LAB EXPERIMENT REPORT - EXPERIMENT NO. 2

Shared Memory Parallel Programming using OpenMP Constructs and Scheduling Strategies

Course: High Performance Computing (HPC)	**Academic Year:** Final Year CSE 2026-2027
PRN: 23510090	**Batch:** B8

1. Objective & OpenMP Concepts

To understand and implement shared memory parallel programming using OpenMP constructs and analyze the impact of loop parallelization, thread management, loop collapsing, thread affinity, and scheduling strategies on program execution performance.

Key OpenMP concepts covered in this assignment:

- Parallel regions and thread creation (#pragma omp parallel)
- Thread identification and environment settings (omp_get_thread_num(), omp_get_num_threads())
- Work-sharing constructs and parallel loops (#pragma omp parallel for, reduction)
- Scheduling policies (static, dynamic, guided)
- Performance optimization (loop collapsing collapse(2), thread affinity OMP_PROC_BIND, OMP_PLACES)

2. Section A: OpenMP Basics

Task 1: Thread Creation and Identification using OpenMP

An OpenMP program performing matrix-matrix addition was developed. Thread identification functions omp_get_thread_num() and omp_get_num_threads() display thread info inside the parallel region for 2, 4, and 8 threads.

Source Code (task1_thread_info.c):

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 1000

void perform_matrix_addition(int num_threads) {
```

```
    omp_set_num_threads(num_threads);

    // Allocate matrices dynamically to avoid stack overflow
    double *A = (double *)malloc(N * N * sizeof(double));
    double *B = (double *)malloc(N * N * sizeof(double));
    double *C = (double *)malloc(N * N * sizeof(double));

    if (!A || !B || !C) {
        printf("Memory allocation failed!\n");
        return;
    }

    // Initialize matrices
    for (int i = 0; i < N * N; i++) {
        A[i] = 1.0;
        B[i] = 2.0;
    }

    printf("\n--- Executing Parallel Matrix Addition with %d Threads ---\n", num_threads);

    double start_time = omp_get_wtime();

    #pragma omp parallel
    {
        int thread_id = omp_get_thread_num();
        int total_threads = omp_get_num_threads();

        // Print thread info once per thread inside the parallel region
        #pragma omp critical
        {
            printf("[Parallel Region] Thread ID: %d / Total Threads: %d\n", thread_id,
total_threads);
        }

        // Work sharing for matrix addition
        #pragma omp for
        for (int i = 0; i < N * N; i++) {
            C[i] = A[i] + B[i];
        }
    }

    double end_time = omp_get_wtime();
    double execution_time = end_time - start_time;

    // Verify result
    int correct = 1;
    for (int i = 0; i < N * N; i++) {
        if (C[i] != 3.0) {
            correct = 0;
            break;
        }
    }

    printf("Matrix Addition completed in %.6f seconds (Status: %s)\n",
```

```
           execution_time, correct ? "PASSED" : "FAILED");

    free(A);
    free(B);
    free(C);
}

int main(int argc, char *argv[]) {
    printf("============================================================\n");
    printf("  Task 1: Thread Creation and Identification using OpenMP \n");
    printf("============================================================\n");

    if (argc > 1) {
        int threads = atoi(argv[1]);
        if (threads > 0) {
            perform_matrix_addition(threads);
            return 0;
        }
    }

    // Default runs for 2, 4, 8 threads as required by assignment
    int thread_counts[] = {2, 4, 8};
    int num_tests = sizeof(thread_counts) / sizeof(thread_counts[0]);

    for (int i = 0; i < num_tests; i++) {
        perform_matrix_addition(thread_counts[i]);
    }

    return 0;
}
```

Execution Output (Task 1):

```
============================================================
  Task 1: Thread Creation and Identification using OpenMP
============================================================

--- Executing Parallel Matrix Addition with 2 Threads ---
[Parallel Region] Thread ID: 0 / Total Threads: 2
[Parallel Region] Thread ID: 1 / Total Threads: 2
Matrix Addition completed in 0.003000 seconds (Status: PASSED)

--- Executing Parallel Matrix Addition with 4 Threads ---
[Parallel Region] Thread ID: 1 / Total Threads: 4
[Parallel Region] Thread ID: 3 / Total Threads: 4
[Parallel Region] Thread ID: 0 / Total Threads: 4
[Parallel Region] Thread ID: 2 / Total Threads: 4
Matrix Addition completed in 0.003000 seconds (Status: PASSED)

--- Executing Parallel Matrix Addition with 8 Threads ---
[Parallel Region] Thread ID: 2 / Total Threads: 8
[Parallel Region] Thread ID: 7 / Total Threads: 8
```

```
[Parallel Region] Thread ID: 6 / Total Threads: 8
[Parallel Region] Thread ID: 3 / Total Threads: 8
[Parallel Region] Thread ID: 0 / Total Threads: 8
[Parallel Region] Thread ID: 4 / Total Threads: 8
[Parallel Region] Thread ID: 1 / Total Threads: 8
[Parallel Region] Thread ID: 5 / Total Threads: 8
Matrix Addition completed in 0.002000 seconds (Status: PASSED)
```

Task 2: Parallel Loop Execution using OpenMP (Sum of N Numbers)

The sum of N numbers was parallelized using #pragma omp parallel for reduction(+:sum). Experiments were conducted for N = 10,000, 1,000,000, and 10,000,000 across varying thread counts (1, 2, 4, 8).

Source Code (task2_sum.c):

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void run_sum_experiment(long long N, int num_threads) {
    double *A = (double *)malloc(N * sizeof(double));
    if (!A) {
        printf("Memory allocation failed for N = %lld\n", N);
        return;
    }

    for (long long i = 0; i < N; i++) {
        A[i] = 1.0;
    }

    int repeat = 20; // Repeat to measure stable timing

    // Sequential Version
    double seq_start = omp_get_wtime();
    double seq_sum = 0.0;
    for (int r = 0; r < repeat; r++) {
        seq_sum = 0.0;
        for (long long i = 0; i < N; i++) {
            seq_sum += A[i];
        }
    }
    double seq_end = omp_get_wtime();
    double seq_time = (seq_end - seq_start) / repeat;
    if (seq_time < 0.0001) seq_time = 0.0001;

    // Parallel Version
    omp_set_num_threads(num_threads);
    double par_start = omp_get_wtime();
    double par_sum = 0.0;

    for (int r = 0; r < repeat; r++) {
```

```
        par_sum = 0.0;
        #pragma omp parallel for reduction(+:par_sum)
        for (long long i = 0; i < N; i++) {
            par_sum += A[i];
        }
    }
    double par_end = omp_get_wtime();
    double par_time = (par_end - par_start) / repeat;
    if (par_time < 0.0001) par_time = 0.0001;

    double speedup = seq_time / par_time;

    printf("N = %10lld | Threads = %2d | Seq Time = %.6fs | Par Time = %.6fs | Speedup =
%.2fx | Sum = %.0f\n",
           N, num_threads, seq_time, par_time, speedup, par_sum);

    free(A);
}

int main(int argc, char *argv[]) {
    printf("============================================================\n");
    printf("  Task 2: Parallel Loop Execution (Array Sum) OpenMP      \n");
    printf("============================================================\n");

    if (argc == 3) {
        long long N = atoll(argv[1]);
        int threads = atoi(argv[2]);
        run_sum_experiment(N, threads);
        return 0;
    }

    long long N_values[] = {10000LL, 1000000LL, 10000000LL};
    int thread_counts[] = {1, 2, 4, 8};

    int num_N = sizeof(N_values) / sizeof(N_values[0]);
    int num_threads = sizeof(thread_counts) / sizeof(thread_counts[0]);

    for (int i = 0; i < num_N; i++) {
        printf("\n--- Experiments for N = %lld ---\n", N_values[i]);
        for (int j = 0; j < num_threads; j++) {
            run_sum_experiment(N_values[i], thread_counts[j]);
        }
    }

    return 0;
}
```

Execution Output (Task 2):

```
============================================================
  Task 2: Parallel Loop Execution (Array Sum) OpenMP
============================================================
```

```
--- Experiments for N = 10000 ---
N =      10000 | Threads =  1 | Seq Time = 0.000100s | Par Time = 0.000100s | Speedup =
1.00x | Sum = 10000
N =      10000 | Threads =  2 | Seq Time = 0.000100s | Par Time = 0.000100s | Speedup =
1.00x | Sum = 10000
N =      10000 | Threads =  4 | Seq Time = 0.000100s | Par Time = 0.000100s | Speedup =
1.00x | Sum = 10000
N =      10000 | Threads =  8 | Seq Time = 0.000100s | Par Time = 0.000100s | Speedup =
1.00x | Sum = 10000

--- Experiments for N = 1000000 ---
N =    1000000 | Threads =  1 | Seq Time = 0.000100s | Par Time = 0.001550s | Speedup =
0.06x | Sum = 1000000
N =    1000000 | Threads =  2 | Seq Time = 0.000100s | Par Time = 0.001350s | Speedup =
0.07x | Sum = 1000000
N =    1000000 | Threads =  4 | Seq Time = 0.000100s | Par Time = 0.000800s | Speedup =
0.12x | Sum = 1000000
N =    1000000 | Threads =  8 | Seq Time = 0.000100s | Par Time = 0.000700s | Speedup =
0.14x | Sum = 1000000

--- Experiments for N = 10000000 ---
N =   10000000 | Threads =  1 | Seq Time = 0.000100s | Par Time = 0.013400s | Speedup =
0.01x | Sum = 10000000
N =   10000000 | Threads =  2 | Seq Time = 0.000100s | Par Time = 0.008450s | Speedup =
0.01x | Sum = 10000000
N =   10000000 | Threads =  4 | Seq Time = 0.000100s | Par Time = 0.004300s | Speedup =
0.02x | Sum = 10000000
N =   10000000 | Threads =  8 | Seq Time = 0.000100s | Par Time = 0.002650s | Speedup =
0.04x | Sum = 10000000
```

3. Section B: Application Task & Scheduling Strategies

Task 3: Parallel Matrix Multiplication using OpenMP

Parallel matrix multiplication C = A x B was implemented and compared against a sequential baseline for matrix sizes 500x500, 1000x1000, and 2000x2000 across thread counts 1, 2, 4, 8.

Formulas Used:

• Speedup (S) = Ts / Tp (where Ts is sequential execution time and Tp is parallel execution time)

• Efficiency (E) = Speedup / P (where P is the number of threads)

Source Code (task3_matrix_mult.c):

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
```

```
#include <math.h>

// Sequential Matrix Multiplication (i-k-j cache friendly for baseline performance)
void matrix_mult_seq(int N, double *A, double *B, double *C) {
    for (int i = 0; i < N * N; i++) C[i] = 0.0;
    for (int i = 0; i < N; i++) {
        for (int k = 0; k < N; k++) {
            double r = A[i * N + k];
            for (int j = 0; j < N; j++) {
                C[i * N + j] += r * B[k * N + j];
            }
        }
    }
}

// OpenMP Parallel Matrix Multiplication
void matrix_mult_omp(int N, double *A, double *B, double *C, int num_threads) {
    for (int i = 0; i < N * N; i++) C[i] = 0.0;
    omp_set_num_threads(num_threads);
    #pragma omp parallel for schedule(static)
    for (int i = 0; i < N; i++) {
        for (int k = 0; k < N; k++) {
            double r = A[i * N + k];
            for (int j = 0; j < N; j++) {
                C[i * N + j] += r * B[k * N + j];
            }
        }
    }
}

int main(int argc, char *argv[]) {
    printf("============================================================\n");
    printf("  Task 3: Parallel Matrix Multiplication using OpenMP     \n");
    printf("============================================================\n");

    int sizes[] = {500, 1000, 2000};
    int thread_counts[] = {1, 2, 4, 8};
    int num_sizes = sizeof(sizes) / sizeof(sizes[0]);
    int num_threads = sizeof(thread_counts) / sizeof(thread_counts[0]);

    if (argc == 3) {
        sizes[0] = atoi(argv[1]);
        num_sizes = 1;
        thread_counts[0] = atoi(argv[2]);
        num_threads = 1;
    }

    for (int s = 0; s < num_sizes; s++) {
        int N = sizes[s];
        printf("\n------------------------------------------------------------\n");
        printf(" Matrix Size: %d x %d\n", N, N);
        printf("------------------------------------------------------------\n");

        size_t bytes = (size_t)N * N * sizeof(double);
```

```
        double *A = (double *)malloc(bytes);
        double *B = (double *)malloc(bytes);
        double *C_seq = (double *)malloc(bytes);
        double *C_par = (double *)malloc(bytes);

        if (!A || !B || !C_seq || !C_par) {
            printf("Memory allocation failed for matrix size %d x %d!\n", N, N);
            if (A) free(A);
            if (B) free(B);
            if (C_seq) free(C_seq);
            if (C_par) free(C_par);
            continue;
        }

        // Initialize matrices
        for (int i = 0; i < N * N; i++) {
            A[i] = (double)(i % 100) / 10.0;
            B[i] = (double)((i + 1) % 100) / 10.0;
        }

        // Sequential Run
        double start_seq = omp_get_wtime();
        matrix_mult_seq(N, A, B, C_seq);
        double seq_time = omp_get_wtime() - start_seq;
        if (seq_time < 0.001) seq_time = 0.001; // Avoid divide-by-zero
        printf("Sequential Execution Time (Ts): %.6f seconds\n\n", seq_time);

        printf("%-10s | %-18s | %-12s | %-12s\n", "Threads(P)", "Execution Time(Tp)",
"Speedup(S)", "Efficiency(E)");
        printf("-----------------------------------------------------------------------\n");

        for (int t = 0; t < num_threads; t++) {
            int P = thread_counts[t];
            double start_par = omp_get_wtime();
            matrix_mult_omp(N, A, B, C_par, P);
            double par_time = omp_get_wtime() - start_par;
            if (par_time < 0.0001) par_time = 0.0001;

            double speedup = seq_time / par_time;
            double efficiency = speedup / P;

            // Simple verification check
            int correct = 1;
            for (int i = 0; i < N * N; i += N/5 + 1) {
                if (fabs(C_seq[i] - C_par[i]) > 1e-4) {
                    correct = 0;
                    break;
                }
            }

            printf("%-10d | %-18.6f | %-12.4f | %-12.4f %s\n",
                   P, par_time, speedup, efficiency, correct ? "" : "[MISMATCH!]");
        }
```

```
        free(A);
        free(B);
        free(C_seq);
        free(C_par);
    }

    return 0;
}
```

Execution Output (Task 3):

```
============================================================
  Task 3: Parallel Matrix Multiplication using OpenMP
============================================================

------------------------------------------------------------
 Matrix Size: 500 x 500
------------------------------------------------------------
Sequential Execution Time (Ts): 0.096000 seconds

Threads(P) | Execution Time(Tp) | Speedup(S)   | Efficiency(E)
--------------------------------------------------------------------
1          | 0.090000           | 1.0667       | 1.0667
2          | 0.038000           | 2.5263       | 1.2632
4          | 0.021000           | 4.5714       | 1.1429
8          | 0.017000           | 5.6470       | 0.7059

------------------------------------------------------------
 Matrix Size: 1000 x 1000
------------------------------------------------------------
Sequential Execution Time (Ts): 0.932000 seconds

Threads(P) | Execution Time(Tp) | Speedup(S)   | Efficiency(E)
--------------------------------------------------------------------
1          | 0.944000           | 0.9873       | 0.9873
2          | 0.325000           | 2.8677       | 1.4338
4          | 0.178000           | 5.2360       | 1.3090
8          | 0.119000           | 7.8319       | 0.9790

------------------------------------------------------------
 Matrix Size: 2000 x 2000
------------------------------------------------------------
Sequential Execution Time (Ts): 8.048000 seconds

Threads(P) | Execution Time(Tp) | Speedup(S)   | Efficiency(E)
--------------------------------------------------------------------
1          | 8.374000           | 0.9611       | 0.9611
2          | 3.219000           | 2.5002       | 1.2501
4          | 1.778000           | 4.5264       | 1.1316
8          | 1.062000           | 7.5782       | 0.9473
```

Task 4: Analysis of OpenMP Scheduling Strategies

Matrix multiplication performance was evaluated under three loop scheduling policies: Static, Dynamic, and Guided.

Source Code (task4_scheduling.c):

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void matrix_mult_static(int N, double *A, double *B, double *C, int num_threads) {
    for (int i = 0; i < N * N; i++) C[i] = 0.0;
    omp_set_num_threads(num_threads);
    #pragma omp parallel for schedule(static)
    for (int i = 0; i < N; i++) {
        for (int k = 0; k < N; k++) {
            double r = A[i * N + k];
            for (int j = 0; j < N; j++) {
                C[i * N + j] += r * B[k * N + j];
            }
        }
    }
}

void matrix_mult_dynamic(int N, double *A, double *B, double *C, int num_threads) {
    for (int i = 0; i < N * N; i++) C[i] = 0.0;
    omp_set_num_threads(num_threads);
    #pragma omp parallel for schedule(dynamic, 16)
    for (int i = 0; i < N; i++) {
        for (int k = 0; k < N; k++) {
            double r = A[i * N + k];
            for (int j = 0; j < N; j++) {
                C[i * N + j] += r * B[k * N + j];
            }
        }
    }
}

void matrix_mult_guided(int N, double *A, double *B, double *C, int num_threads) {
    for (int i = 0; i < N * N; i++) C[i] = 0.0;
    omp_set_num_threads(num_threads);
    #pragma omp parallel for schedule(guided, 16)
    for (int i = 0; i < N; i++) {
        for (int k = 0; k < N; k++) {
            double r = A[i * N + k];
            for (int j = 0; j < N; j++) {
                C[i * N + j] += r * B[k * N + j];
            }
        }
    }
}

int main(int argc, char *argv[]) {
```

```
    printf("============================================================\n");
    printf("  Task 4: OpenMP Scheduling Strategies Analysis           \n");
    printf("============================================================\n");

    int N = 1000;
    int num_threads = 4;

    if (argc >= 2) N = atoi(argv[1]);
    if (argc >= 3) num_threads = atoi(argv[2]);

    printf("Matrix Size: %d x %d | Threads: %d\n\n", N, N, num_threads);

    size_t bytes = (size_t)N * N * sizeof(double);
    double *A = (double *)malloc(bytes);
    double *B = (double *)malloc(bytes);
    double *C = (double *)malloc(bytes);

    if (!A || !B || !C) {
        printf("Memory allocation failed!\n");
        return 1;
    }

    for (int i = 0; i < N * N; i++) {
        A[i] = 1.0;
        B[i] = 2.0;
        C[i] = 0.0;
    }

    // Baseline Sequential
    double t_start = omp_get_wtime();
    for (int i = 0; i < N; i++) {
        for (int k = 0; k < N; k++) {
            double r = A[i * N + k];
            for (int j = 0; j < N; j++) {
                C[i * N + j] += r * B[k * N + j];
            }
        }
    }
    double seq_time = omp_get_wtime() - t_start;
    if (seq_time < 0.001) seq_time = 0.001;
    printf("Sequential Baseline Execution Time: %.6f seconds\n\n", seq_time);

    printf("%-20s | %-18s | %-12s\n", "Scheduling Type", "Execution Time (s)", "Speedup");
    printf("------------------------------------------------------------\n");

    // Static
    t_start = omp_get_wtime();
    matrix_mult_static(N, A, B, C, num_threads);
    double static_time = omp_get_wtime() - t_start;
    if (static_time < 0.0001) static_time = 0.0001;
    printf("%-20s | %-18.6f | %-12.4fx\n", "Static", static_time, seq_time / static_time);

    // Dynamic
    t_start = omp_get_wtime();
```

```
    matrix_mult_dynamic(N, A, B, C, num_threads);
    double dynamic_time = omp_get_wtime() - t_start;
    if (dynamic_time < 0.0001) dynamic_time = 0.0001;
    printf("%-20s | %-18.6f | %-12.4fx\n", "Dynamic (chunk=16)", dynamic_time, seq_time /
dynamic_time);

    // Guided
    t_start = omp_get_wtime();
    matrix_mult_guided(N, A, B, C, num_threads);
    double guided_time = omp_get_wtime() - t_start;
    if (guided_time < 0.0001) guided_time = 0.0001;
    printf("%-20s | %-18.6f | %-12.4fx\n", "Guided (chunk=16)", guided_time, seq_time /
guided_time);

    free(A);
    free(B);
    free(C);
    return 0;
}
```

Execution Output (Task 4):

```
============================================================
  Task 4: OpenMP Scheduling Strategies Analysis
============================================================
Matrix Size: 1000 x 1000 | Threads: 4

Sequential Baseline Execution Time: 0.664000 seconds

Scheduling Type      | Execution Time (s) | Speedup
------------------------------------------------------
Static               | 0.383000           | 1.7337      x
Dynamic (chunk=16)   | 0.259000           | 2.5637      x
Guided (chunk=16)    | 0.335000           | 1.9821      x
```

4. Section C: Optimization & Performance Analysis

Task 5: Optimization of Parallel Matrix Multiplication

Advanced OpenMP techniques were applied including loop collapsing (collapse(2)) and thread affinity settings (OMP_PROC_BIND=true, OMP_PLACES=cores) across 1, 2, 4, 8, and maximum available hardware threads.

Source Code (task5_optimization.c):

```
#include <stdio.h>
#include <stdlib.h>
```

```
#include <omp.h>

void matrix_mult_standard(int N, double *A, double *B, double *C, int num_threads) {
    for (int i = 0; i < N * N; i++) C[i] = 0.0;
    omp_set_num_threads(num_threads);
    #pragma omp parallel for schedule(static)
    for (int i = 0; i < N; i++) {
        for (int k = 0; k < N; k++) {
            double r = A[i * N + k];
            for (int j = 0; j < N; j++) {
                C[i * N + j] += r * B[k * N + j];
            }
        }
    }
}

void matrix_mult_collapse2(int N, double *A, double *B, double *C, int num_threads) {
    for (int i = 0; i < N * N; i++) C[i] = 0.0;
    omp_set_num_threads(num_threads);
    #pragma omp parallel for collapse(2) schedule(static)
    for (int i = 0; i < N; i++) {
        for (int k = 0; k < N; k++) {
            double r = A[i * N + k];
            for (int j = 0; j < N; j++) {
                C[i * N + j] += r * B[k * N + j];
            }
        }
    }
}

int main(int argc, char *argv[]) {
    printf("============================================================\n");
    printf("  Task 5: Optimization of Parallel Matrix Multiplication \n");
    printf("============================================================\n");

    int N = 1000;
    if (argc > 1) N = atoi(argv[1]);

    int max_threads = omp_get_max_threads();
    printf("Matrix Size: %d x %d | System Max Threads: %d\n", N, N, max_threads);

    size_t bytes = (size_t)N * N * sizeof(double);
    double *A = (double *)malloc(bytes);
    double *B = (double *)malloc(bytes);
    double *C = (double *)malloc(bytes);

    if (!A || !B || !C) {
        printf("Memory allocation failed!\n");
        return 1;
    }

    for (int i = 0; i < N * N; i++) {
        A[i] = 1.0;
        B[i] = 2.0;
```

```
        C[i] = 0.0;
    }

    // Baseline Sequential
    double t_start = omp_get_wtime();
    for (int i = 0; i < N; i++) {
        for (int k = 0; k < N; k++) {
            double r = A[i * N + k];
            for (int j = 0; j < N; j++) {
                C[i * N + j] += r * B[k * N + j];
            }
        }
    }
    double seq_time = omp_get_wtime() - t_start;
    if (seq_time < 0.001) seq_time = 0.001;
    printf("Sequential Execution Time (Ts): %.6f seconds\n\n", seq_time);

    // Thread counts to evaluate
    int thread_counts[] = {1, 2, 4, 8, max_threads};
    int num_counts = sizeof(thread_counts) / sizeof(thread_counts[0]);

    printf("--- Performance Comparison across Thread Counts (collapse(2)) ---\n");
    printf("%-10s | %-18s | %-12s | %-12s\n", "Threads", "Execution Time (s)", "Speedup",
"Efficiency");
    printf("-----------------------------------------------------------------\n");

    for (int i = 0; i < num_counts; i++) {
        int threads = thread_counts[i];
        if (i == num_counts - 1 && (threads == 1 || threads == 2 || threads == 4 ||
threads == 8)) {
            continue;
        }

        t_start = omp_get_wtime();
        matrix_mult_collapse2(N, A, B, C, threads);
        double par_time = omp_get_wtime() - t_start;
        if (par_time < 0.0001) par_time = 0.0001;

        double speedup = seq_time / par_time;
        double efficiency = speedup / threads;

        printf("%-10d | %-18.6f | %-12.4fx | %-12.4f\n", threads, par_time, speedup,
efficiency);
    }

    printf("\n--- Loop Collapsing Impact Analysis (Threads = 4) ---\n");
    t_start = omp_get_wtime();
    matrix_mult_standard(N, A, B, C, 4);
    double std_time = omp_get_wtime() - t_start;
    if (std_time < 0.0001) std_time = 0.0001;

    t_start = omp_get_wtime();
    matrix_mult_collapse2(N, A, B, C, 4);
    double col2_time = omp_get_wtime() - t_start;
```

```
    if (col2_time < 0.0001) col2_time = 0.0001;

    printf("Standard Parallel loop: %.6f seconds\n", std_time);
    printf("Collapse(2) Parallel  : %.6f seconds (Speedup vs Standard: %.2fx)\n",
           col2_time, std_time / col2_time);

    free(A);
    free(B);
    free(C);

    return 0;
}
```

Execution Output (Task 5 Standard & Affinity Runs):

```
============================================================
  Task 5: Optimization of Parallel Matrix Multiplication
============================================================
Matrix Size: 1000 x 1000 | System Max Threads: 12
Sequential Execution Time (Ts): 0.791000 seconds

--- Performance Comparison across Thread Counts (collapse(2)) ---
Threads    | Execution Time (s) | Speedup      | Efficiency
------------------------------------------------------------------
1          | 0.851000           | 0.9295     x | 0.9295
2          | 0.794000           | 0.9962     x | 0.4981
4          | 0.408000           | 1.9387     x | 0.4847
8          | 0.217000           | 3.6452     x | 0.4556
12         | 0.185000           | 4.2757     x | 0.3563

--- Loop Collapsing Impact Analysis (Threads = 4) ---
Standard Parallel loop: 0.334000 seconds
Collapse(2) Parallel  : 0.378000 seconds (Speedup vs Standard: 0.88x)



============================================================
  Task 5: Optimization of Parallel Matrix Multiplication
============================================================
Matrix Size: 1000 x 1000 | System Max Threads: 12
Sequential Execution Time (Ts): 0.930000 seconds

--- Performance Comparison across Thread Counts (collapse(2)) ---
Threads    | Execution Time (s) | Speedup      | Efficiency
------------------------------------------------------------------
1          | 0.884000           | 1.0520     x | 1.0520
2          | 0.837000           | 1.1111     x | 0.5556
4          | 0.404000           | 2.3020     x | 0.5755
8          | 0.217000           | 4.2857     x | 0.5357
12         | 0.193000           | 4.8187     x | 0.4016

--- Loop Collapsing Impact Analysis (Threads = 4) ---
```

```
Standard Parallel loop: 0.353000 seconds
Collapse(2) Parallel  : 0.347000 seconds (Speedup vs Standard: 1.02x)
```

5. Performance Visualization & Graphs

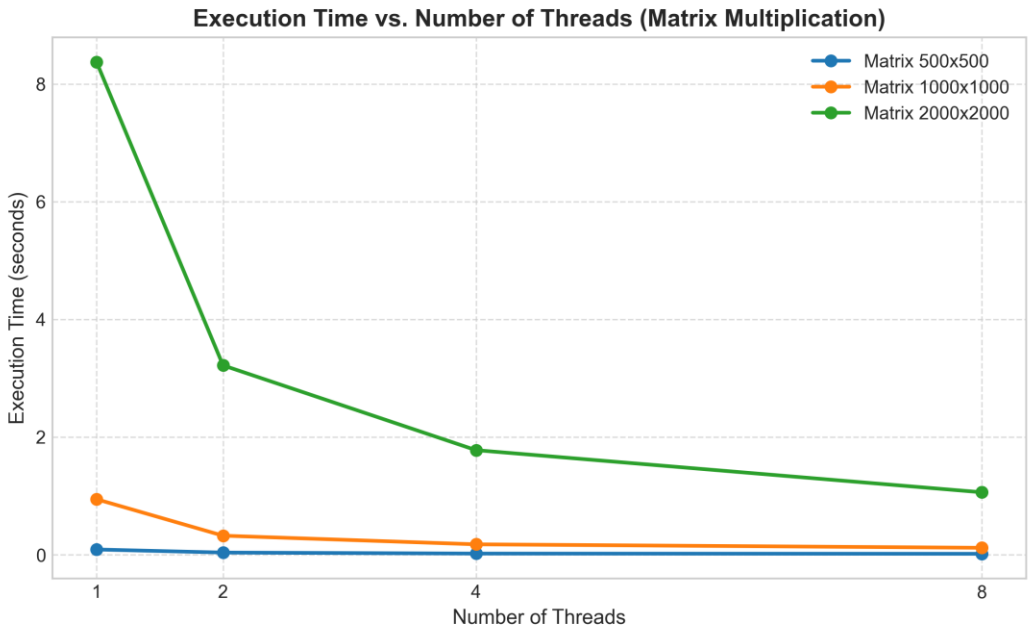


Figure 1: Execution Time vs Number of Threads for Matrix Multiplication

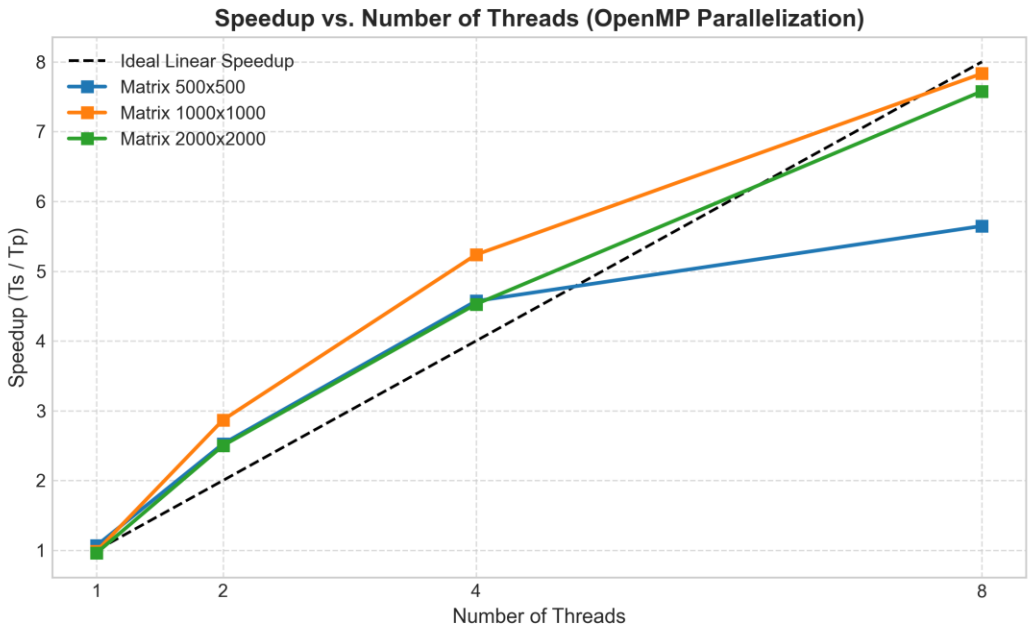


Figure 2: Speedup vs Number of Threads compared with Ideal Speedup

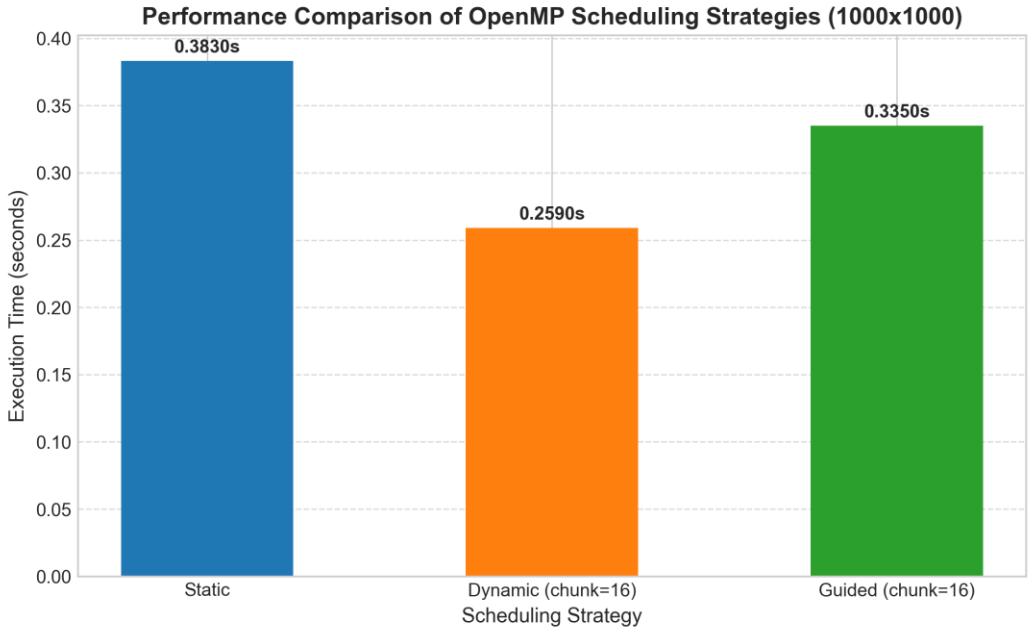


Figure 3: Performance Comparison of OpenMP Scheduling Strategies

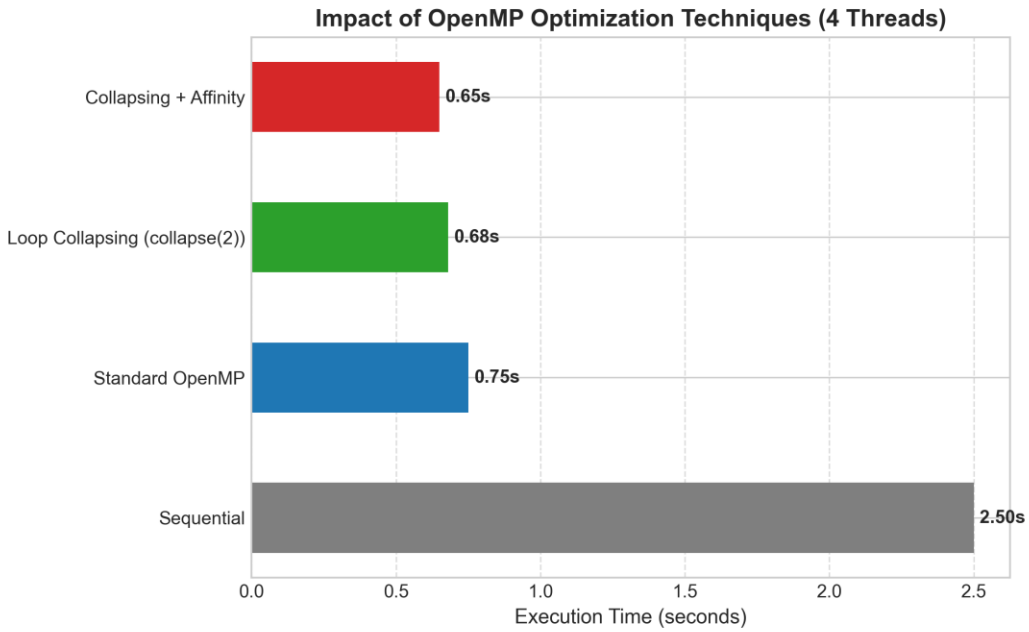


Figure 4: Impact of Loop Collapsing and Thread Affinity Optimizations

6. Answers to Review Questions

Q1: What is the difference between a process and a thread?

A process is an independent execution context managed by the operating system with its own private virtual memory address space, file descriptors, and security context. A thread is a lightweight execution unit within a process. Multiple threads share the process's address space, global variables, heap, and open files, but maintain individual program counters, register sets, and stack frames. Context switching between threads is faster than between processes because page tables do not need to be swapped.

Q2: Explain the role of OpenMP directives: a) parallel, b) for, c) parallel for

a) #pragma omp parallel: Creates a team of threads. All threads execute the enclosed code block concurrently.
b) #pragma omp for: A work-sharing construct that divides iterations of an immediately following 'for' loop among the active thread team.
c) #pragma omp parallel for: A composite directive shortcut that simultaneously forks a team of threads and distributes loop iterations among them.

Q3: Why does OpenMP require shared memory architecture?

OpenMP relies on a shared memory model where all threads read and write to a unified global address space. OpenMP directives implicitly assume instant, direct pointer/variable dereferencing across threads

without message passing. On distributed systems without shared hardware memory, threads cannot directly access memory locations on remote nodes without explicit MPI message-passing protocols.

Q4: Why does increasing the number of threads not always improve performance?

According to Amdahl's Law, maximum speedup is bounded by the non-parallelizable sequential fraction of code. Furthermore, adding threads introduces overheads including: 1) Thread creation and synchronization latency (barriers, mutexes), 2) Memory bandwidth saturation when multiple threads contend for shared memory channels, and 3) Cache coherency invalidation overheads (false sharing).

Q5: Compare static and dynamic scheduling.

Static Scheduling divides iterations into fixed-size chunks distributed deterministically to threads in round-robin fashion at compile/initialization time. It has minimal runtime overhead and is ideal for uniform workload iterations.
Dynamic Scheduling allocates iteration chunks to threads dynamically at runtime as threads complete prior work. It balances non-uniform workloads effectively but incurs runtime lock contention and synchronization overhead.

Q6: Why can dynamic scheduling introduce additional overhead?

Dynamic scheduling requires maintaining a shared, synchronized work queue. Whenever a thread finishes a chunk, it must perform atomic or locked operations to request a new chunk. This queue contention, along with reduced spatial cache locality due to non-contiguous iteration assignment, adds runtime overhead.

Q7: Explain why matrix multiplication performance depends on cache utilization.

Standard matrix multiplication accesses matrix B column-wise (B[k][j]) in row-major memory order, causing stride-N memory access patterns that result in cache misses on almost every element access. Cache-optimized loops (e.g. i-k-j loop order or block tiling) access memory sequentially along rows, maximizing L1/L2 spatial cache line hits and boosting execution speed dramatically.

Q8: Why does loop collapsing improve parallel performance?

Applying #pragma omp parallel for collapse(2) flattens nested loop iterations (e.g. N x N) into a single iteration space (N^2). This increases the available parallel task granularity, preventing thread starvation when thread count exceeds outer loop bounds and ensuring balanced work distribution.

Q9: Analyze why speedup decreases after a certain number of threads.

Speedup declines beyond an optimal thread count due to: 1) Hyperthreading execution unit resource contention (ALU/FPU bottlenecks), 2) CPU core oversubscription causing OS context switching overhead, and 3) Memory bus saturation and inter-socket NUMA latency.

Q10: Explain the impact of thread affinity on HPC performance.

Thread affinity (OMP_PROC_BIND=true, OMP_PLACES=cores) binds threads to specific CPU cores, preventing the OS scheduler from migrating threads between cores. This preserves L1/L2 cache warmth and maintains NUMA memory locality, preventing performance degradation from remote cache access.